1

Desarrollo de Software 1 · Semana 1

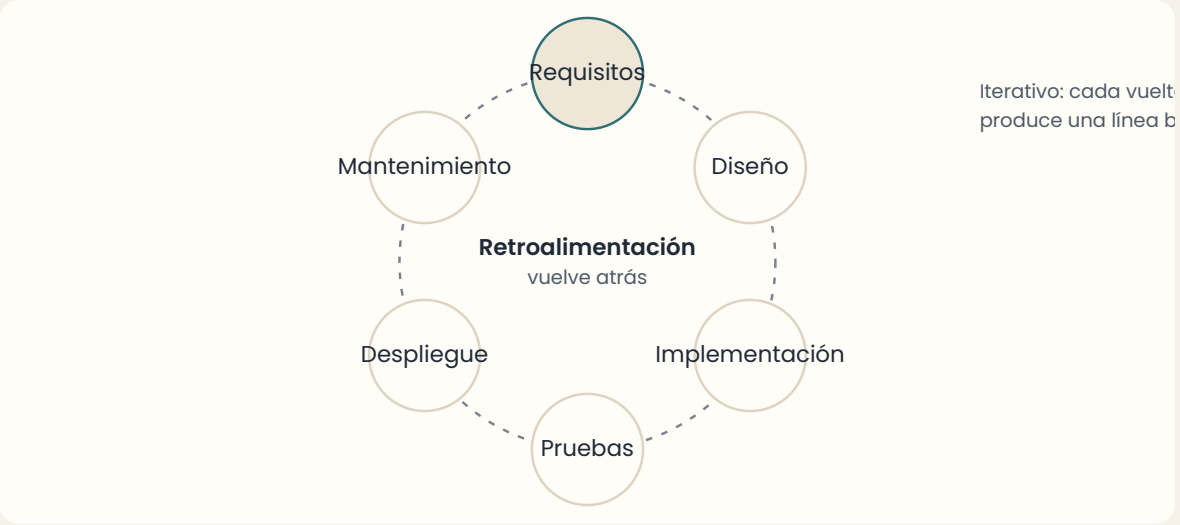
Introducción a la gestión de la ingeniería de software

La ingeniería de software trata de entregar y mantener software, no solo de escribirlo.

- La mayoría de los problemas de proyecto vienen de brechas de comunicación y planificación.
- Los modelos permiten discutir y verificar un sistema antes de construirlo.

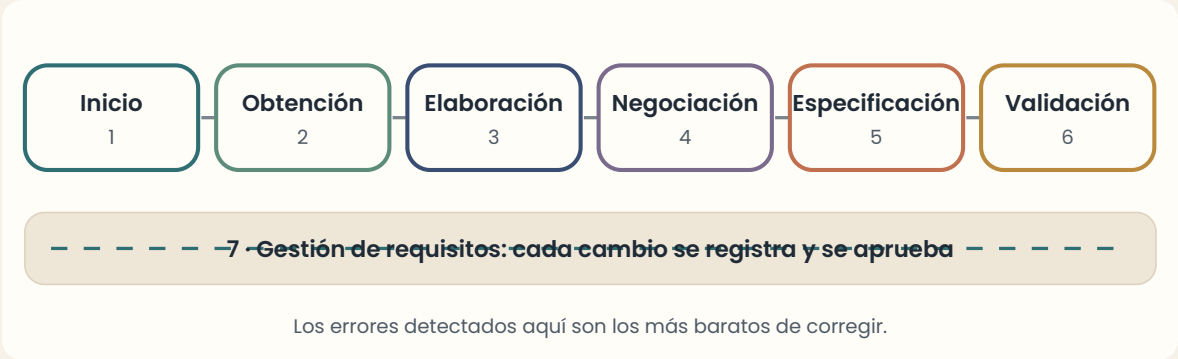
> **Piénsalo así.** Construir una casa necesita planos antes que ladrillos. El software necesita planes y modelos antes que código, porque cambiar un muro en el papel cuesta mucho menos que derribarlo.

Un modelo de proceso es una descripción simplificada de cómo trabajan los equipos reales.



> **Piénsalo así.** Piensa en una clase de cocina. Cascada es seguir una receta de principio a fin sin probar. El trabajo iterativo es probar la salsa después de cada paso y ajustar el siguiente.

Un requisito debe ser comprobable: alguien debe poder decir "se cumple" o "no se cumple".



> **Piénsalo así.** Un sastre mide dos veces y corta una. Los requisitos son las medidas: si están mal, ninguna costura cuidadosa (programación) hará que el traje quede bien.

Un modelo es útil cuando es más simple que el sistema pero sigue siendo correcto en lo que importa.



> **Piénsalo así.** El mapa de una ciudad es un modelo. Ignora el color de cada casa (abstracto) pero acierta con las calles (preciso), así que puedes planear una ruta antes de caminar (predictivo y barato).

Los elementos de configuración incluyen todo lo que forma el producto, no solo el código.

- La GCS responde: qué versión, qué cambio, aprobado por quién y en qué línea base.

> **Piénsalo así.** Una biblioteca mantiene un catálogo de cada edición y de quién la prestó. La GCS hace lo mismo con un producto: cada elemento, versión y cambio tiene un registro que puedes consultar.

Un VCS guarda el historial, permite recuperar versiones y muestra quién cambió qué y cuándo.

- Git es distribuido; GitHub es un servicio de alojamiento para repositorios Git.
- Un commit debe contener un solo cambio lógico con un mensaje claro.

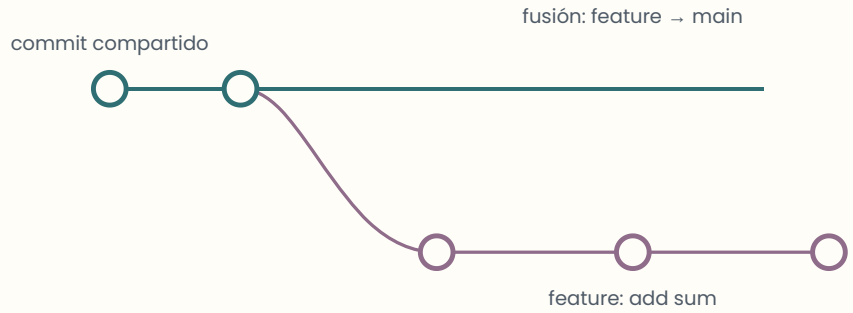
> **Piénsalo así.** El control de versiones es como un videojuego con puntos de guardado. Puedes volver a cualquier punto, ver qué cambió entre dos y dejar que tus amigos jueguen desde su propia copia.

Edita en el directorio de trabajo, prepara con git add y registra con git commit.



> **Piénsalo así.** Git es un estudio fotográfico. El directorio de trabajo es el set, la preparación es elegir qué fotos van al álbum, y el commit es imprimir la página del álbum. Una rama es un segundo álbum que puedes armar sin tocar el primero.

origin es el nombre habitual del remoto que clonaste o conectaste.



Cada punto es un commit. Las ramas son punteros; fusionar combina el historial.

> **Piénsalo así.** Una solicitud de cambio es como un borrador enviado a un compañero antes de entregar el trabajo. El revisor no lo escribe por ti, pero detecta errores mientras son fáciles de corregir.

Paso a paso

Pon el ciclo de vida en orden

1. Análisis de requisitos
2. Diseño
3. Implementación
4. Pruebas
5. Despliegue
6. Mantenimiento

Ponte a prueba

1. ¿Qué elemento es un elemento de configuración en la gestión de la configuración del software?

A) Un documento de requisitos B) La cafetera de la oficina C) Un calendario personal

2. ¿Qué afirmación sobre Git es correcta?

A) Git es un sistema de control de versiones distribuido; cada clon contiene el historial completo. B) Git no puede registrar un commit sin una conexión a internet a GitHub. C) Git y GitHub son dos nombres para la misma herramienta.

3. En un proceso iterativo e incremental, ¿qué representa la diferencia entre dos líneas base sucesivas?

A) Un incremento B) Un registro de riesgos C) Solo la entrega final

Las respuestas están en las notas del orador.

Practica después

Laboratorios

- Inicializar un repositorio y hacer tu primer commit
- Leer el historial y trabajar en una rama
- Resolver un conflicto de fusión
- Conectar un remoto y simular una revisión

En el sitio

- Simulador de Git